

【手順書】WebView2を用いた超軽量（155KB）デスクトップウィジェット作成ガイド

- 内部サーバーで公開しているWebページ（[index.html](#)）を、Windows 10/11標準の **WebView2（WPF）** を利用して、背景透過・ドラッグ移動・位置サイズ記憶対応の独立したWindows用デスクトップアプリ（[.exe](#)）として構築する手順です。

1. アプリの主な仕様

- 超軽量動作**: アプリサイズわずか約155KB（Node.js等のインストール不要）
- 背景完全透過**: デスクトップの壁紙が透けるアクリル風デザイン
- 直感的な操作**: 画面の余白やつまみ領域を掴んで滑らかにドラッグ移動
- 四辺リサイズ**: 外枠8px領域を掴んで自由自在にウィンドウサイズを変更
- 自動記憶**: 変更した配置位置（x, y）とサイズ（width, height）を次回起動時に自動復元
- クリーン表示**: リンクホバー時の画面最下部URL表示（ステータスバー）を完全非表示

2. ディレクトリ構成

開発作業フォルダ（例: [WidgetApp](#)）の中に以下のファイルを用意します。

```
WidgetApp/
├─ app.ico           # アイコン画像（.ico形式）
├─ WidgetApp.csproj  # プロジェクト設定ファイル
├─ App.xaml          # WPFアプリケーション定義
├─ App.xaml.cs       # エラー制御処理
├─ MainWindow.xaml   # ウィンドウレイアウト（XAML）
└─ MainWindow.xaml.cs # ロジック（透過、ドラッグ移動、位置記憶等）
```

3. 各ファイルコード

① WidgetApp.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <UseWPF>true</UseWPF>
    <ImplicitUsings>enable</ImplicitUsings>
    <!-- .exe ファイル自体のアイコン指定 -->
    <ApplicationIcon>app.ico</ApplicationIcon>
    <!-- 不要なビルド警告を無効化 -->
    <NoWarn>$(NoWarn);MSB3277;CS8600;CS8622</NoWarn>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Web.WebView2" Version="1.0.*" />
  </ItemGroup>
</Project>
```

② App.xaml

```
<Application x:Class="WidgetApp.App"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  StartupUri="MainWindow.xaml">
</Application>
```

③ App.xaml.cs

```
using System.Windows;

namespace WidgetApp
{
    public partial class App : Application
    {
        public App()
        {
            // 予期せぬエラー発生時にダイアログ表示する安全保護
            this.DispatcherUnhandledException += (s, e) =>
            {
                MessageBox.Show($"起動エラーが発生しました:\n{e.Exception.Message}", "エラー", MessageBoxButton.OK,
                MessageBoxImage.Error);
                e.Handled = true;
            };
        }
    }
}
```

④ MainWindow.xaml

```
<Window x:Class="WidgetApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:wpf="clr-namespace:Microsoft.Web.WebView2.Wpf;assembly=Microsoft.Web.WebView2.Wpf"
        Title="フォルダウィジェット"
        WindowStyle="None"
        AllowsTransparency="True"
        Background="#01000000"
        ResizeMode="CanResize"
        Width="380" Height="600"
        WindowStartupLocation="Manual"
        Loaded="Window_Loaded">
```

```
Closing="Window_Closing"
LocationChanged="Window_LocationChanged"
SizeChanged="Window_SizeChanged">

<!-- 外枠8pxをつかんで自由自在にリサイズできる設定 -->
<WindowChrome.WindowChrome>
    <WindowChrome CaptionHeight="0" ResizeBorderThickness="8" GlassFrameThickness="0" CornerRadius="0"/>
</WindowChrome.WindowChrome>

<Grid Background="#01000000">
    <!-- ★ Margin="6" を指定してリサイズ枠線領域を露出させる -->
    <wv2:WebView2 x:Name="webView" Margin="6" HorizontalAlignment="Stretch" VerticalAlignment="Stretch"/>
</Grid>
</Window>
```

⑤ MainWindow.xaml.cs

```
using System;
using System.ComponentModel;
using System.IO;
using System.Runtime.InteropServices;
using System.Text.Json;
using System.Windows;
using System.Windows.Interop;

namespace WidgetApp
{
    public class WindowStateData
    {
        public double Left { get; set; }
        public double Top { get; set; }
        public double Width { get; set; } = 380;
        public double Height { get; set; } = 600;
    }
}
```

```
public partial class MainWindow : Window
{
    private string configPath;
    private bool isLoading = false;

    // Win32 API: 確実にウィンドウをドラッグ移動させる処理
    [DllImport("user32.dll")]
    private static extern bool ReleaseCapture();

    [DllImport("user32.dll")]
    private static extern IntPtr SendMessage(IntPtr hWnd, int Msg, IntPtr wParam, IntPtr lParam);

    private const int WM_NCLBUTTONDOWN = 0xA1;
    private const int HTCAPTION = 0x2;

    public MainWindow()
    {
        InitializeComponent();

        string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        string folder = Path.Combine(appData, "WidgetApp");
        Directory.CreateDirectory(folder);
        configPath = Path.Combine(folder, "window-state.json");

        LoadWindowState();
    }

    private async void Window_Loaded(object sender, RoutedEventArgs e)
    {
        try
        {
            await webView.EnsureCoreWebView2Async();

            // 1. 最下部リンクポップアップ（ステータスバー）を非表示
            webView.CoreWebView2.Settings.IsStatusBarEnabled = false;
        }
    }
}
```

```
// 2. WebView2の背景透過設定
webView.DefaultBackgroundColor = System.Drawing.Color.Transparent;

// 3. Web画面からのドラッグ移動メッセージを受信
webView.WebMessageReceived += WebView_WebMessageReceived;

// 4. HTML側に目に見えない極小透過 (rgba 0.01) とドラッグ監視JSを自動注入
await webView.CoreWebView2.AddScriptToExecuteOnDocumentCreatedAsync(@"
    window.addEventListener('DOMContentLoaded', () => {
        // 見た目は完全透明だがマウスクリックを透過させない設定
        document.body.style.backgroundColor = 'rgba(0, 0, 0, 0.01)';

        const dragTargets = document.querySelectorAll('.widget-title, body');
        dragTargets.forEach(el => {
            el.addEventListener('mousedown', (e) => {
                if (!e.target.closest('a, button, input, summary, details, .folder-link')) {
                    window.chrome.webview.postMessage('start_drag');
                }
            });
        });
    });
");

// ★表示したい GitHub Pages の URL に書き換えてください
string targetUrl = "https://<ユーザー名>.github.io/<リポジトリ名>/index.html";

if (Uri.TryCreate(targetUrl, UriKind.Absolute, out Uri? uriResult) && uriResult != null)
{
    webView.Source = uriResult;
}
else
{
    MessageBox.Show("URLの形式が正しくありません。URLを確認してください。", "設定エラー");
}
```

```
        isLoading = true;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"WebView2初期化エラー:\n{ex.Message}", "エラー");
    }
}

// Web画面のドラッグ操作をWindowsのネイティブ移動処理へ変換
private void WebView_WebMessageReceived(object? sender,
Microsoft.Web.WebView2.Core.CoreWebView2WebMessageReceivedEventArgs e)
{
    if (e.TryGetWebMessageAsString() == "start_drag")
    {
        var hwnd = new WindowInteropHelper(this).Handle;
        ReleaseCapture();
        SendMessage(hwnd, WM_NCLBUTTONDOWN, (IntPtr)HTCAPTION, IntPtr.Zero);
    }
}

private void LoadWindowState()
{
    try
    {
        if (File.Exists(configPath))
        {
            string json = File.ReadAllText(configPath);
            var state = JsonSerializer.Deserialize<WindowStateData>(json);
            if (state != null)
            {
                this.Width = state.Width >= 150 ? state.Width : 380;
                this.Height = state.Height >= 150 ? state.Height : 600;
                this.Left = state.Left;
                this.Top = state.Top;
                return;
            }
        }
    }
}
```

```
        }
    }
    catch { }

    this.Width = 380;
    this.Height = 600;
}

private void SaveWindowState()
{
    if (!isLoading || this.WindowState != WindowState.Normal) return;

    try
    {
        var state = new WindowStateData
        {
            Left = this.Left,
            Top = this.Top,
            Width = this.Width,
            Height = this.Height
        };
        string json = JsonSerializer.Serialize(state);
        File.WriteAllText(configPath, json);
    }
    catch { }
}

private void Window_LocationChanged(object sender, EventArgs e) => SaveWindowState();
private void Window_SizeChanged(object sender, SizeChangedEventArgs e) => SaveWindowState();
private void Window_Closing(object sender, CancelEventArgs e) => SaveWindowState();
}
}
```

4. ビルド・実行手順

1. コマンドプロンプトで作業フォルダに移動します。

```
cd C:\Path\To\WidgetApp
```

2. 実行中の古いアプリがあれば終了させます。

```
taskkill /f /im WidgetApp.exe
```

3. パブリッシュコマンドを実行します。

```
dotnet publish -c Release
```

4. **出力場所:** `bin\Release\net8.0-windows\publish\WidgetApp.exe` に超軽量（155KB）の実行ファイルが完成します。